Lecture 9. Lazy Evaluation in Haskell

Programming Paradigms, Fall 2026 — 21 Oct 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. Demand and Pattern Matching
3. Lazy Data, Lists, and Trees
4. Folds and Space Use

Where We Are

This Lecture

Topic 8: values that describe input and output.

Topic 9: how much of a pure expression must be evaluated?

1. Predict which arguments and fields an expression needs.
2. Trace a lazy tree passed between two separate functions.
3. Connect a producer and a consumer with a lazy list.
4. Choose between `foldr`, `foldl`, and `foldl'`.

Load `lecture-9.hs` in GHCi to run the examples.

Quiz 8: Naming and Running an `IO` Value



```haskell
hello :: IO ()
hello = putStrLn "hello"
```

```
ghci> let greeting = hello
ghci> greeting
hello
```

`putStrLn :: String -> IO ()` is an ordinary function.
Applying it builds an action; GHCi runs an action entered at its prompt.

Quiz 8: Naming and Running an `IO` Value



```haskell
hello :: IO ()
hello = putStrLn "hello"
```

```
ghci> let greeting = hello
ghci> greeting
hello
```

`putStrLn :: String -> IO ()` is an ordinary function.
Applying it builds an action; GHCi runs an action entered at its prompt.

No `do` is needed for one action.
Running `greeting` again prints the line again.

Quiz 8: What Does <- Bind?



```haskell
import Text.Read (readMaybe)

nextTicket :: String -> Maybe Int
nextTicket s = do
  n <- readMaybe s    -- readMaybe s :: Maybe Int; n :: Int
  Just (n + 1)
```

```
ghci> nextTicket "41"
ghci> nextTicket "bad"
```

Quiz 8: What Does <- Bind?



```haskell
import Text.Read (readMaybe)

nextTicket :: String -> Maybe Int
nextTicket s = do
  n <- readMaybe s    -- readMaybe s :: Maybe Int; n :: Int
  Just (n + 1)
```

```
ghci> nextTicket "41"   Just 42
ghci> nextTicket "bad"
```

Quiz 8: What Does <- Bind?



```haskell
import Text.Read (readMaybe)

nextTicket :: String -> Maybe Int
nextTicket s = do
  n <- readMaybe s    -- readMaybe s :: Maybe Int; n :: Int
  Just (n + 1)
```

```
ghci> nextTicket "41"   Just 42
ghci> nextTicket "bad"  Nothing
```

Here **do** uses the bind operation of **Maybe**:

- **Just** `n` passes `n` to the rest of the block;
- **Nothing** skips the rest and returns **Nothing**.

Quiz 8: Handling `Maybe` Inside `IO`

```haskell
showNext :: String -> IO ()
showNext s = do
  case readMaybe s :: Maybe Int of
    Nothing -> putStrLn "not a ticket number"
    Just n  -> print (n + 1)
  putStrLn "ready"
```

Question

What does `showNext "bad"` print?

Quiz 8: Handling `Maybe` Inside `IO`

```haskell
showNext :: String -> IO ()
showNext s = do
  case readMaybe s :: Maybe Int of
    Nothing -> putStrLn "not a ticket number"
    Just n  -> print (n + 1)
  putStrLn "ready"
```

Question

What does `showNext "bad"` print?

```
not a ticket number
ready
```

Both branches produce `IO ()`, so the block continues.

Quiz 8: Mapping a Pure Function



The function `(+1)` has an ordinary number as its result.

```
map  (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) (Just 41)    -- Just 42
fmap (+1) Nothing      -- Nothing
```

On lists, `fmap` is `map`.

On **`Maybe`**, it changes the value in **`Just`** and keeps **`Nothing`**.

Quiz 8: Mapping a Pure Function



The function `(+1)` has an ordinary number as its result.

```
map  (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) (Just 41)    -- Just 42
fmap (+1) Nothing      -- Nothing
```

On lists, `fmap` is `map`.

On **`Maybe`**, it changes the value in **`Just`** and keeps **`Nothing`**.

```
nextTicket s = fmap (+1) (readMaybe s)
```

Quiz 8: Mapping a Pure Function

The function `(+1)` has an ordinary number as its result.

```
map  (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) (Just 41)    -- Just 42
fmap (+1) Nothing      -- Nothing
```

On lists, `fmap` is `map`.
On **`Maybe`**, it changes the value in **`Just`** and keeps **`Nothing`**.

```
nextTicket s = fmap (+1) (readMaybe s)
```

With bind, the next function returns a **`Maybe`** and can also fail.

Quiz 8: Mapping a Pure Function

The function `(+1)` has an ordinary number as its result.

```
map  (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) [1, 2, 3]    -- [2,3,4]
fmap (+1) (Just 41)    -- Just 42
fmap (+1) Nothing      -- Nothing
```

On lists, `fmap` is `map`.
On **`Maybe`**, it changes the value in **`Just`** and keeps **`Nothing`**.

```
nextTicket s = fmap (+1) (readMaybe s)
```

With bind, the next function returns a **`Maybe`** and can also fail.

Calling `pure` does not return early from a **`do`** block.

Why Delay Evaluation?

We want the caller to decide how much of a computation is needed.

[1] Hughes. Why Functional Programming Matters. 1989, Section 4.

Why Delay Evaluation?

We want the caller to decide how much of a computation is needed.

- Skip unused work: a conditional evaluates only the selected branch.

[1] Hughes. Why Functional Programming Matters. 1989, Section 4.

Why Delay Evaluation?

We want the caller to decide how much of a computation is needed.

- Skip unused work: a conditional evaluates only the selected branch.
- Separate generation from use: a producer describes candidates, and a consumer decides when to stop[1].

[1] Hughes. Why Functional Programming Matters. 1989, Section 4.

Why Delay Evaluation?

We want the caller to decide how much of a computation is needed.

- Skip unused work: a conditional evaluates only the selected branch.
- Separate generation from use: a producer describes candidates, and a consumer decides when to stop[1].
- Work with unbounded structures: inspect a list prefix or a tree branch without building the whole structure.

[1] Hughes. Why Functional Programming Matters. 1989, Section 4.

Why Delay Evaluation?

We want the caller to decide how much of a computation is needed.

- Skip unused work: a conditional evaluates only the selected branch.
- Separate generation from use: a producer describes candidates, and a consumer decides when to stop[1].
- Work with unbounded structures: inspect a list prefix or a tree branch without building the whole structure.

Note

> Laziness helps us compose independently written functions.
> The consumer's demand controls the producer's work.

[1] Hughes. Why Functional Programming Matters. 1989, Section 4.

The Costs of Laziness

Delaying a computation changes when its work and errors become visible.

The Costs of Laziness

Delaying a computation changes when its work and errors become visible.

- A small result may still need unbounded work: a search can examine candidates forever without finding a match.

The Costs of Laziness

Delaying a computation changes when its work and errors become visible.

- A small result may still need unbounded work: a search can examine candidates forever without finding a match.
- Unevaluated computations take space and can retain their inputs. Sharing can keep data alive for later uses.

The Costs of Laziness

Delaying a computation changes when its work and errors become visible.

- A small result may still need unbounded work: a search can examine candidates forever without finding a match.
- Unevaluated computations take space and can retain their inputs. Sharing can keep data alive for later uses.
- An unused field can hide an error until a consumer demands it.

The Costs of Laziness

Delaying a computation changes when its work and errors become visible.

- A small result may still need unbounded work: a search can examine candidates forever without finding a match.
- Unevaluated computations take space and can retain their inputs. Sharing can keep data alive for later uses.
- An unused field can hide an error until a consumer demands it.

Pitfall

> Laziness does not promise termination or lower memory use.
> We must ask what is demanded and what remains reachable.

Demand and Pattern Matching

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

1. Try the first equation, whose first pattern is `True`.

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

1. Try the first equation, whose first pattern is `True`.
2. Evaluate `3 < 2` to decide whether it matches: `False`.

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

1. Try the first equation, whose first pattern is `True`.
2. Evaluate `3 < 2` to decide whether it matches: `False`.
3. The first equation fails; try the second equation.

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

1. Try the first equation, whose first pattern is `True`.
2. Evaluate `3 < 2` to decide whether it matches: `False`.
3. The first equation fails; try the second equation.
4. `False` matches. Bind `x` and `y` without evaluating them.

Lazy Function Application

```haskell
f :: Bool -> Int -> Int -> Int
f True  x y = x
f False x y = y
```

Suppose we ask GHCi for `f (3 < 2) (length [1, 2, 3]) 0`.

1. Try the first equation, whose first pattern is `True`.
2. Evaluate `3 < 2` to decide whether it matches: `False`.
3. The first equation fails; try the second equation.
4. `False` matches. Bind `x` and `y` without evaluating them.
5. Return `y`, which is `0`. The `length` call is unused.

A User-Defined Conditional

```
choose :: Bool -> a -> a -> a
choose True  x _ = x
choose False _ y = y
```

We can write a conditional as an ordinary function.

```
choose (3 < 2) (length [1, 2, 3]) 0   -- 0
choose True    (length [1, 2, 3]) 0   -- 3
```

A User-Defined Conditional

```
choose :: Bool -> a -> a -> a
choose True  x _ = x
choose False _ y = y
```

We can write a conditional as an ordinary function.

```
choose (3 < 2) (length [1, 2, 3]) 0   -- 0
choose True    (length [1, 2, 3]) 0   -- 3
```

Note

Matching the variables does not evaluate the branches.
Demanding the result then demands the selected branch.

Testing Demand

`undefined :: a` raises an error when its value is demanded[2].

```
choose False undefined 0   -- 0
choose True  3 undefined   -- 3
choose True  undefined 0   -- error
choose undefined 3 0       -- error
```

A function is **strict** in an argument if an undefined argument makes the result undefined, for every choice of the other arguments.

[2] Marlow (ed.). Haskell 2010 Language Report. 2010, Sections 3.1 and 3.17.

Testing Demand

`undefined :: a` raises an error when its value is demanded[2].

```
choose False undefined 0   -- 0
choose True  3 undefined   -- 3
choose True  undefined 0   -- error
choose undefined 3 0       -- error
```

A function is **strict** in an argument if an undefined argument makes the result undefined, for every choice of the other arguments.

`choose` is strict in its condition.
Neither branch is always demanded; the condition decides which one is.

[2] Marlow (ed.). Haskell 2010 Language Report. 2010, Sections 3.1 and 3.17.

Laziness and Sharing

```haskell
twice :: Int -> Int
twice x = x + x

twice (length [1, 2, 3])
```

Laziness and Sharing

```haskell
twice :: Int -> Int
twice x = x + x

twice (length [1, 2, 3])
```

1. **Call by name** delays the argument but may recompute it at each use.

Laziness and Sharing

```haskell
twice :: Int -> Int
twice x = x + x

twice (length [1, 2, 3])
```

1. **Call by name** delays the argument but may recompute it at each use.
2. **Call by need** delays the argument and shares its result.

Laziness and Sharing

```haskell
twice :: Int -> Int
twice x = x + x

twice (length [1, 2, 3])
```

1. **Call by name** delays the argument but may recompute it at each use.
2. **Call by need** delays the argument and shares its result.
3. In this model, both uses of `x` share one delayed `length` call.

Laziness and Sharing

```haskell
twice :: Int -> Int
twice x = x + x

twice (length [1, 2, 3])
```

1. **Call by name** delays the argument but may recompute it at each use.
2. **Call by need** delays the argument and shares its result.
3. In this model, both uses of `x` share one delayed `length` call.

Haskell's semantics are **non-strict**: an unused argument may be undefined.
We use lazy evaluation as our execution model; GHC may optimise it.

Logical And: A Truth Table

```haskell
andTable :: Bool -> Bool -> Bool
andTable False False = False
andTable False True  = False
andTable True  False = False
andTable True  True  = True
```

Question

What happens on `andTable False undefined`?

Logical And: A Truth Table

```haskell
andTable :: Bool -> Bool -> Bool
andTable False False = False
andTable False True  = False
andTable True  False = False
andTable True  True  = True
```

Question

What happens on `andTable False undefined`?

An error: the first equation demands the second argument.
This definition is strict in both arguments.

Logical And: A Truth Table

```haskell
andTable :: Bool -> Bool -> Bool
andTable False False = False
andTable False True  = False
andTable True  False = False
andTable True  True  = True
```

Question

What happens on `andTable False undefined`?

An error: the first equation demands the second argument.
This definition is strict in both arguments.

Can we return `False` without inspecting the second argument?

Logical And: Combining Two Cases

```
andCases :: Bool -> Bool -> Bool
andCases False _     = False
andCases True  False = False
andCases True  True  = True
```

```
ghci>  andCases False undefined
```

Logical And: Combining Two Cases

```haskell
andCases :: Bool -> Bool -> Bool
andCases False _     = False
andCases True  False = False
andCases True  True  = True
```

```
ghci>  andCases False undefined
False
```

A wildcard `_` matches without inspecting its argument.
The first argument still decides which equation applies.

Logical And: Combining Two Cases

```haskell
andCases :: Bool -> Bool -> Bool
andCases False _     = False
andCases True  False = False
andCases True  True  = True
```

```
ghci>  andCases False undefined
False
```

A wildcard `_` matches without inspecting its argument.
The first argument still decides which equation applies.

Can we combine the remaining two cases?

Logical And: Returning the Second Argument

```haskell
andLeft :: Bool -> Bool -> Bool
andLeft False _ = False
andLeft _     y = y
```

This has the same demand behaviour as the Prelude's `(&&)`[3].

[3] Marlow (ed.). Haskell 2010 Language Report. 2010, Chapter 9.

Logical And: Returning the Second Argument

```haskell
andLeft :: Bool -> Bool -> Bool
andLeft False _ = False
andLeft _     y = y
```

This has the same demand behaviour as the Prelude's `(&&)`[3].

```haskell
andLeft False undefined
```

[3] Marlow (ed.). Haskell 2010 Language Report. 2010, Chapter 9.

Logical And: Returning the Second Argument

```haskell
andLeft :: Bool -> Bool -> Bool
andLeft False _ = False
andLeft _     y = y
```

This has the same demand behaviour as the Prelude's `(&&)`[3].

```haskell
andLeft False undefined
= False
```

[3] Marlow (ed.). Haskell 2010 Language Report. 2010, Chapter 9.

Logical And: Returning the Second Argument

```
andLeft :: Bool -> Bool -> Bool
andLeft False _ = False
andLeft _     y = y
```

This has the same demand behaviour as the Prelude's `(&&)`[3].

```
andLeft False undefined
= False
```

For `andLeft True undefined`, the selected result is undefined.

[3] Marlow (ed.). Haskell 2010 Language Report. 2010, Chapter 9.

Logical And: Choosing the Other Argument First

```haskell
andRight :: Bool -> Bool -> Bool
andRight _ False = False
andRight x _     = x
```

Compare the two definitions:

Arguments	`andLeft`	`andRight`
`False`, `undefined`		
`undefined`, **`False`**		

Logical And: Choosing the Other Argument First

```
andRight :: Bool -> Bool -> Bool
andRight _ False = False
andRight x _     = x
```

Compare the two definitions:

Arguments	`andLeft`	`andRight`
`False`, `undefined`	**`False`**	error
`undefined`, **`False`**		

Logical And: Choosing the Other Argument First

```
andRight :: Bool -> Bool -> Bool
andRight _ False = False
andRight x _     = x
```

Compare the two definitions:

Arguments	`andLeft`	`andRight`
`False`, `undefined`	**`False`**	error
`undefined`, **`False`**	error	**`False`**

Logical And: Choosing the Other Argument First

```
andRight :: Bool -> Bool -> Bool
andRight _ False = False
andRight x _     = x
```

Compare the two definitions:

Arguments	`andLeft`	`andRight`
`False`, `undefined`	**`False`**	error
`undefined`, **`False`**	error	**`False`**

The truth tables agree on defined Booleans.
Their behaviour on undefined arguments differs.

Lazy Data, Lists, and Trees

Constructors and Their Fields

```haskell
type Name = String
type Grade = Double
data Student = Student Name Grade

nameOf :: Student -> Name
nameOf (Student name _) = name
```

```haskell
nameOf (Student "Yoko" 4.5)          -- "Yoko"
nameOf (Student "Yoko" undefined)    -- "Yoko"
```

Matching `Student` reveals the constructor.
It does not demand the grade.
Ordinary `data` fields are lazy by default.

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	
`Just (1 + 2)`	
`1 : undefined`	
`1 + 2 :: Int`	
`undefined :: [Int]`	

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	
`1 : undefined`	
`1 + 2 :: Int`	
`undefined :: [Int]`	

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	yes: `Just`
`1 : undefined`	
`1 + 2 :: Int`	
`undefined :: [Int]`	

How Much Has Been Evaluated?



Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	yes: `Just`
`1 : undefined`	yes: `(:)`
`1 + 2 :: Int`	
`undefined :: [Int]`	

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	yes: `Just`
`1 : undefined`	yes: `(:)`
`1 + 2 :: Int`	no: the addition is still to be evaluated
`undefined :: [Int]`	

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	yes: `Just`
`1 : undefined`	yes: `(:)`
`1 + 2 :: Int`	no: the addition is still to be evaluated
`undefined :: [Int]`	no: it cannot produce a constructor

How Much Has Been Evaluated?

Weak head normal form (WHNF): the outer constructor or function is exposed; its contents may still be unevaluated.

Expression	Already in WHNF?
`Student "Yoko" undefined`	yes: `Student`
`Just (1 + 2)`	yes: `Just`
`1 : undefined`	yes: `(:)`
`1 + 2 :: Int`	no: the addition is still to be evaluated
`undefined :: [Int]`	no: it cannot produce a constructor

Seeing a list's first constructor is less work than printing the whole list.

A List Has a Spine and Elements

```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [1, undefined, 4]
```

A List Has a Spine and Elements

```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [1, undefined, 4]
= 1 + length [undefined, 4]
```

A List Has a Spine and Elements

```
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```
length [1, undefined, 4]
= 1 + length [undefined, 4]
= 1 + (1 + length [4])
```

A List Has a Spine and Elements



```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [1, undefined, 4]
= 1 + length [undefined, 4]
= 1 + (1 + length [4])
= 1 + (1 + (1 + length []))
```

A List Has a Spine and Elements



```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [1, undefined, 4]
= 1 + length [undefined, 4]
= 1 + (1 + length [4])
= 1 + (1 + (1 + length []))
= 1 + (1 + (1 + 0))
```

A List Has a Spine and Elements



```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [1, undefined, 4]
= 1 + length [undefined, 4]
= 1 + (1 + length [4])
= 1 + (1 + (1 + length []))
= 1 + (1 + (1 + 0))
= 3
```

A List Has a Spine and Elements

```
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```
length [1, undefined, 4]
= 1 + length [undefined, 4]
= 1 + (1 + length [4])
= 1 + (1 + (1 + length []))
= 1 + (1 + (1 + 0))
= 3
```

We inspect the `(:)` and `[]` constructors (the **spine**), but never the elements.

Question: Which Part is Missing?

```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [undefined]              -- A
length (1 : undefined)          -- B
length (undefined :: [Int])     -- C
```

Question

Which calls return a number, and which raise an error?

Question: Which Part is Missing?

```haskell
length :: [a] -> Int
length []     = 0
length (_:xs) = 1 + length xs
```

```haskell
length [undefined]              -- A
length (1 : undefined)          -- B
length (undefined :: [Int])     -- C
```

Question

Which calls return a number, and which raise an error?

A returns `1`. B and C raise an error.
An undefined element is different from an undefined tail.

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
```

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
```

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
= 1 : (2 : take 1 [3, 4, 5])
```

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
= 1 : (2 : take 1 [3, 4, 5])
= 1 : (2 : (3 : take 0 [4, 5]))
```

Taking a Prefix



```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
= 1 : (2 : take 1 [3, 4, 5])
= 1 : (2 : (3 : take 0 [4, 5]))
= [1, 2, 3]
```

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
= 1 : (2 : take 1 [3, 4, 5])
= 1 : (2 : (3 : take 0 [4, 5]))
= [1, 2, 3]
```

Why must the first equation come before inspecting the list?

Taking a Prefix

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 3 [1, 2, 3, 4, 5]
= 1 : take 2 [2, 3, 4, 5]
= 1 : (2 : take 1 [3, 4, 5])
= 1 : (2 : (3 : take 0 [4, 5]))
= [1, 2, 3]
```

Why must the first equation come before inspecting the list?

`take 0 undefined` must return `[]`.

The Outer Consumer Decides

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 1 (take 3 [1, 2, 3, 4, 5])
```

The Outer Consumer Decides

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 1 (take 3 [1, 2, 3, 4, 5])
= take 1 (1 : take 2 [2, 3, 4, 5])
```

The Outer Consumer Decides



```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 1 (take 3 [1, 2, 3, 4, 5])
= take 1 (1 : take 2 [2, 3, 4, 5])
= 1 : take 0 (take 2 [2, 3, 4, 5])
```

The Outer Consumer Decides



```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 1 (take 3 [1, 2, 3, 4, 5])
= take 1 (1 : take 2 [2, 3, 4, 5])
= 1 : take 0 (take 2 [2, 3, 4, 5])
= 1 : []
```

The Outer Consumer Decides

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```haskell
take 1 (take 3 [1, 2, 3, 4, 5])
= take 1 (1 : take 2 [2, 3, 4, 5])
= 1 : take 0 (take 2 [2, 3, 4, 5])
= 1 : []
= [1]
```

The Outer Consumer Decides

```haskell
take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ []         = []
take n (x:xs)     = x : take (n - 1) xs
```

```
take 1 (take 3 [1, 2, 3, 4, 5])
= take 1 (1 : take 2 [2, 3, 4, 5])
= 1 : take 0 (take 2 [2, 3, 4, 5])
= 1 : []
= [1]
```

The inner call produces only the part the outer call needs.

Generating a Finite Range

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi   = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
```

Generating a Finite Range

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi    = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
= 1 : fromTo 2 3
```

Generating a Finite Range

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi   = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
= 1 : fromTo 2 3
= 1 : 2 : fromTo 3 3
```

Generating a Finite Range

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi   = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
= 1 : fromTo 2 3
= 1 : 2 : fromTo 3 3
= 1 : 2 : 3 : fromTo 4 3
```

Generating a Finite Range

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi   = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
= 1 : fromTo 2 3
= 1 : 2 : fromTo 3 3
= 1 : 2 : 3 : fromTo 4 3
= [1, 2, 3]
```

Generating a Finite Range



```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi
  | lo > hi   = []
  | otherwise = lo : fromTo (lo + 1) hi
```

```haskell
fromTo 1 3
= 1 : fromTo 2 3
= 1 : 2 : fromTo 3 3
= 1 : 2 : 3 : fromTo 4 3
= [1, 2, 3]
```

`Integer` avoids a bounded machine integer wrapping around.

Generating an Unbounded Range



```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
```

Generating an Unbounded Range

```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
= 1 : take 2 (from 2)
```

Generating an Unbounded Range



```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
= 1 : take 2 (from 2)
= 1 : 2 : take 1 (from 3)
```

Generating an Unbounded Range



```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
= 1 : take 2 (from 2)
= 1 : 2 : take 1 (from 3)
= 1 : 2 : 3 : take 0 (from 4)
```

Generating an Unbounded Range



```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
= 1 : take 2 (from 2)
= 1 : 2 : take 1 (from 3)
= 1 : 2 : 3 : take 0 (from 4)
= [1, 2, 3]
```

Generating an Unbounded Range

```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)
```

```haskell
take 3 (from 1)
= 1 : take 2 (from 2)
= 1 : 2 : take 1 (from 3)
= 1 : 2 : 3 : take 0 (from 4)
= [1, 2, 3]
```

Each demanded cell takes finite work.
An infinite list is never stored in full.

Range Notation

```
[1..5]                  :: [Integer]  -- [1,2,3,4,5]
take 5 [1..]            :: [Integer]  -- [1,2,3,4,5]
[1,3..10]               :: [Integer]  -- [1,3,5,7,9]
take 5 [1,3..]          :: [Integer]  -- [1,3,5,7,9]
[5,4..1]                :: [Integer]  -- [5,4,3,2,1]
```

`[start,next..end]` uses the step `next - start`.

The type matters: `[1..] :: [Int]` ends at `maxBound`.

Range Notation

```
[1..5]                  :: [Integer]  -- [1,2,3,4,5]
take 5 [1..]            :: [Integer]  -- [1,2,3,4,5]
[1,3..10]               :: [Integer]  -- [1,3,5,7,9]
take 5 [1,3..]          :: [Integer]  -- [1,3,5,7,9]
[5,4..1]                :: [Integer]  -- [5,4,3,2,1]
```

`[start,next..end]` uses the step `next - start`.

The type matters: `[1..] :: [Int]` ends at `maxBound`.

We use **`Integer`** for unbounded integer streams.

Task: Stop an Unbounded Producer

```haskell
from :: Integer -> [Integer]
from n = n : from (n + 1)

takeWhile :: (a -> Bool) -> [a] -> [a]
```

`takeWhile p` returns the prefix before the first element that fails `p`.

Two minutes, on paper

Define `fromTo lo hi` using `from` and `takeWhile`.
Check both `fromTo 2 5` and `fromTo 5 2`.

Task: Stop an Unbounded Producer, Solved



```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi = takeWhile (<= hi) (from lo)
```

```
ghci> fromTo 2 5
[2,3,4,5]
ghci> fromTo 5 2
[]
```

Task: Stop an Unbounded Producer, Solved

```haskell
fromTo :: Integer -> Integer -> [Integer]
fromTo lo hi = takeWhile (<= hi) (from lo)
```

```
ghci>  fromTo 2 5
[2,3,4,5]
ghci>  fromTo 5 2
[]
```

Note

The consumer supplies the stopping condition.
When `lo > hi`, even the first candidate fails the test.

Laziness Connects Separate Functions

```haskell
squares :: [Integer]
squares = map (\n -> n * n) [1..]

oddSquares :: [Integer]
oddSquares = filter odd squares

take 4 oddSquares   -- [1,9,25,49]
```

1. The producer describes all the candidates.
2. The filter keeps those that satisfy a condition.
3. The consumer decides how many results it needs.

The producer needs no size parameter.[4]

[4] Hughes. Why Functional Programming Matters. 1989, Section 4.

Laziness Does Not Guarantee Progress

A **productive** stream produces each next constructor in finite time.

```
take 3 (filter odd [1..] :: [Integer])    -- [1,3,5]
take 1 (filter (< 0) [1..] :: [Integer])  -- no result
```

Laziness Does Not Guarantee Progress

A **productive** stream produces each next constructor in finite time.

```
take 3 (filter odd [1..] :: [Integer])    -- [1,3,5]
take 1 (filter (< 0) [1..] :: [Integer])  -- no result
```

The second filter examines candidates forever without finding a match.
Asking for one element need not require only a little work.

Laziness Does Not Guarantee Progress

A **productive** stream produces each next constructor in finite time.

```
take 3 (filter odd [1..] :: [Integer])    -- [1,3,5]
take 1 (filter (< 0) [1..] :: [Integer])  -- no result
```

The second filter examines candidates forever without finding a match.
Asking for one element need not require only a little work.

Pitfall

Even `length ([1..] :: [Integer])` cannot finish.
A finite observation must demand only a finite amount of work.

First Pass: Substitute a Known Value

Suppose we know a variable's integer value.

```haskell
data Expr = Lit Int | Var String
          | Add Expr Expr | Mul Expr Expr

substitute :: String -> Int -> Expr -> Expr
substitute _ _ (Lit m) = Lit m
substitute x n (Var y)
  | x == y    = Lit n
  | otherwise = Var y
substitute x n (Add a b) =
  Add (substitute x n a) (substitute x n b)
substitute x n (Mul a b) =
  Mul (substitute x n a) (substitute x n b)
```

First Pass: Substitute a Known Value

Suppose we know a variable's integer value.

```haskell
data Expr = Lit Int | Var String
          | Add Expr Expr | Mul Expr Expr

substitute :: String -> Int -> Expr -> Expr
substitute _ _ (Lit m) = Lit m
substitute x n (Var y)
  | x == y    = Lit n
  | otherwise = Var y
substitute x n (Add a b) =
  Add (substitute x n a) (substitute x n b)
substitute x n (Mul a b) =
  Mul (substitute x n a) (substitute x n b)
```

Constructing a result node leaves its children unevaluated.

Second Pass: Simplify Zero Products

Use one rule: `0 * e = 0`, for pure, total integer expressions.

```haskell
simplify :: Expr -> Expr
simplify (Lit n) = Lit n
simplify (Var x) = Var x
simplify (Add a b) =
  Add (simplify a) (simplify b)
simplify (Mul a b) =
  mulE (simplify a) (simplify b)
```

```haskell
mulE :: Expr -> Expr -> Expr
mulE (Lit 0) _ = Lit 0
mulE a b = Mul a b
```

Question

Does `mulE (Lit 0) e` inspect `e`?

Second Pass: Simplify Zero Products

Use one rule: `0 * e = 0`, for pure, total integer expressions.

```haskell
simplify :: Expr -> Expr
simplify (Lit n) = Lit n
simplify (Var x) = Var x
simplify (Add a b) =
  Add (simplify a) (simplify b)
simplify (Mul a b) =
  mulE (simplify a) (simplify b)
```

```haskell
mulE :: Expr -> Expr -> Expr
mulE (Lit 0) _ = Lit 0
mulE a b = Mul a b
```

Question

Does `mulE (Lit 0) e` inspect `e`?

No. This rule discards the right subtree without simplifying it.

Two Passes, One Lazy Intermediate Tree

Let `rhs = Add (Var "x") (Lit 1)`.

```
substituted = substitute "x" 0 (Mul (Var "x") rhs)
result      = simplify substituted

result
```

Question

Which pass traverses `rhs`?

Two Passes, One Lazy Intermediate Tree

Let `rhs = Add (Var "x") (Lit 1)`.

```
substituted = substitute "x" 0 (Mul (Var "x") rhs)
result      = simplify substituted
```

```
result
= simplify (Mul (Lit 0) (substitute "x" 0 rhs))
```

Question

Which pass traverses `rhs`?

Two Passes, One Lazy Intermediate Tree

Let `rhs = Add (Var "x") (Lit 1)`.

```
substituted = substitute "x" 0 (Mul (Var "x") rhs)
result      = simplify substituted
```

```
result
= simplify (Mul (Lit 0) (substitute "x" 0 rhs))
= mulE (Lit 0) (simplify (substitute "x" 0 rhs))
```

Question

Which pass traverses `rhs`?

Two Passes, One Lazy Intermediate Tree

Let `rhs = Add (Var "x") (Lit 1)`.

```
substituted = substitute "x" 0 (Mul (Var "x") rhs)
result      = simplify substituted
```

```
result
= simplify (Mul (Lit 0) (substitute "x" 0 rhs))
= mulE (Lit 0) (simplify (substitute "x" 0 rhs))
= Lit 0
```

Question

Which pass traverses `rhs`?

Two Passes, One Lazy Intermediate Tree

Let `rhs = Add (Var "x") (Lit 1)`.

```
substituted = substitute "x" 0 (Mul (Var "x") rhs)
result      = simplify substituted
```

```
result
= simplify (Mul (Lit 0) (substitute "x" 0 rhs))
= mulE (Lit 0) (simplify (substitute "x" 0 rhs))
= Lit 0
```

Question

Which pass traverses `rhs`?

Neither pass does. The intermediate right subtree is never built.

Modular Code, Incremental Execution

Code organisation and subtree demand are separate choices.[5]

Code organisation	Process every subtree	Skip discarded subtrees
One combined function	Always recurse into both children.	Check for zero before recursing on the right.
Separate functions	Build the whole substituted tree first.	Let `simplify` demand parts of the substituted tree.

[5] Kiselyov, Simple generators in OCaml (August 2023), with a tree pipeline adapted here.
[6] Xia et al., Story of Your Lazy Function's Life (ICFP 2024), Section 2.1: cost of `take` with insertion sort.

Modular Code, Incremental Execution

Code organisation and subtree demand are separate choices.[5]

Code organisation	Process every subtree	Skip discarded subtrees
One combined function	Always recurse into both children.	Check for zero before recursing on the right.
Separate functions	Build the whole substituted tree first.	Let `simplify` demand parts of the substituted tree.

Our two-pass pipeline is in the bottom-right cell.

[5] Kiselyov, Simple generators in OCaml (August 2023), with a tree pipeline adapted here.
[6] Xia et al., Story of Your Lazy Function's Life (ICFP 2024), Section 2.1: cost of `take` with insertion sort.

Modular Code, Incremental Execution

Code organisation and subtree demand are separate choices.[5]

Code organisation	Process every subtree	Skip discarded subtrees
One combined function	Always recurse into both children.	Check for zero before recursing on the right.
Separate functions	Build the whole substituted tree first.	Let `simplify` demand parts of the substituted tree.

Our two-pass pipeline is in the bottom-right cell.

Laziness alone does not remove intermediate trees or bound memory use.[6]

[5] Kiselyov, Simple generators in OCaml (August 2023), with a tree pipeline adapted here.

[6] Xia et al., Story of Your Lazy Function's Life (ICFP 2024), Section 2.1: cost of `take` with insertion sort.

5 min break



Folds and Space Use

The Left Fold: A State Passed Along

```haskell
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl _ z []     = z
foldl f z (x:xs) = foldl f (f z x) xs
```

```haskell
foldl (+) 0 [1, 2, 3, 4]   -- 10
foldl (*) 1 [1, 2, 3, 4]   -- 24
```

- `a` is the element type; `b` is the accumulator type.
- `f` takes the accumulator first and the next element second.
- The fold returns the accumulator after reaching `[]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
Initial accumulator: `[]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
After Anna (4.3): `["Anna"]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
After Boris (3.6): `["Anna"]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
After Charles (4.1): `["Charles", "Anna"]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
After Daria (2.9): `["Charles", "Anna"]`.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
Result: `["Charles", "Anna"]`.

Prepending reverses the selected names.

The Left Fold: Selecting Names

```haskell
data Student = Student String Double
students = [Student "Anna" 4.3, Student "Boris" 3.6,
            Student "Charles" 4.1, Student "Daria" 2.9]

step :: [String] -> Student -> [String]
step names (Student name grade)
  | grade > 4.0 = name : names
  | otherwise   = names
```

`foldl step [] students`: follow the accumulator's value.
Result: `["Charles", "Anna"]`.

Prepending reverses the selected names.

These are accumulator values, not a promise of evaluation order.

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs

foldl (+) 0 [1, 2, 3]
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) ((0 + 1) + 2) [3]
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) ((0 + 1) + 2) [3]
= foldl (+) (((0 + 1) + 2) + 3) []
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) ((0 + 1) + 2) [3]
= foldl (+) (((0 + 1) + 2) + 3) []
= ((0 + 1) + 2) + 3
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) ((0 + 1) + 2) [3]
= foldl (+) (((0 + 1) + 2) + 3) []
= ((0 + 1) + 2) + 3
= 6
```

A Tail Call Can Still Accumulate Work

```
foldl f z (x:xs) = foldl f (f z x) xs
```

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) ((0 + 1) + 2) [3]
= foldl (+) (((0 + 1) + 2) + 3) []
= ((0 + 1) + 2) + 3
= 6
```

Pitfall

The lazy model builds a chain of delayed additions.
A tail call alone does not guarantee constant space.

The Strict Left Fold



```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
```

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

The Strict Left Fold



```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
= foldl' (+) 1 [2, 3]
```

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

The Strict Left Fold

```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
= foldl' (+) 1 [2, 3]
= foldl' (+) 3 [3]
```

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

The Strict Left Fold



```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
= foldl' (+) 1 [2, 3]
= foldl' (+) 3 [3]
= foldl' (+) 6 []
```

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

The Strict Left Fold

```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
= foldl' (+) 1 [2, 3]
= foldl' (+) 3 [3]
= foldl' (+) 6 []
= 6
```

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

The Strict Left Fold

```haskell
import Data.List (foldl')
-- Specialised to lists:
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

```haskell
foldl' (+) 0 [1, 2, 3]
= foldl' (+) 1 [2, 3]
= foldl' (+) 3 [3]
= foldl' (+) 6 []
= 6
```

The accumulator is forced to WHNF as the fold proceeds[7].

For `Int`, each sum is evaluated.

A pair can still contain delayed fields.

[7] GHC contributors. Data.List. `base` 4.20.0.0, 2024: `foldl'` and `foldl`.

A Mean in Two Traversals



```haskell
meanTwoPasses :: [Double] -> Maybe Double
meanTwoPasses [] = Nothing
meanTwoPasses xs =
  Just (sum xs / fromIntegral (length xs))
```

[8] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: avoiding multi-pass folds.

A Mean in Two Traversals



```haskell
meanTwoPasses :: [Double] -> Maybe Double
meanTwoPasses [] = Nothing
meanTwoPasses xs =
  Just (sum xs / fromIntegral (length xs))
```

- Both `sum` and `length` consume the entire list.

[8] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: avoiding multi-pass folds.

A Mean in Two Traversals



```haskell
meanTwoPasses :: [Double] -> Maybe Double
meanTwoPasses [] = Nothing
meanTwoPasses xs =
  Just (sum xs / fromIntegral (length xs))
```

- Both `sum` and `length` consume the entire list.
- While one traversal runs, the other still needs the list from its beginning.

[8] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: avoiding multi-pass folds.

A Mean in Two Traversals



```haskell
meanTwoPasses :: [Double] -> Maybe Double
meanTwoPasses [] = Nothing
meanTwoPasses xs =
  Just (sum xs / fromIntegral (length xs))
```

- Both `sum` and `length` consume the entire list.
- While one traversal runs, the other still needs the list from its beginning.
- The visited cells remain reachable, so they cannot be reclaimed yet.

[8] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: avoiding multi-pass folds.

A Mean in Two Traversals

```haskell
meanTwoPasses :: [Double] -> Maybe Double
meanTwoPasses [] = Nothing
meanTwoPasses xs =
  Just (sum xs / fromIntegral (length xs))
```

- Both `sum` and `length` consume the entire list.
- While one traversal runs, the other still needs the list from its beginning.
- The visited cells remain reachable, so they cannot be reclaimed yet.

Sharing saves recomputation, but can retain the entire input[8].

[8] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: avoiding multi-pass folds.

A Strict Fold Can Keep Lazy Fields



```haskell
meanPair :: [Double] -> Maybe Double
meanPair [] = Nothing
meanPair xs = Just (total / fromIntegral count)
  where
    (total, count) = foldl' (\(s, n) x -> (s + x, n + 1))
                            (0, 0 :: Int) xs
```

A Strict Fold Can Keep Lazy Fields



```haskell
meanPair :: [Double] -> Maybe Double
meanPair [] = Nothing
meanPair xs = Just (total / fromIntegral count)
  where
    (total, count) = foldl' (\(s, n) x -> (s + x, n + 1))
                            (0, 0 :: Int) xs
```

One traversal, but after two steps the pair can contain:

```
((0 + x1) + x2, (0 + 1) + 1)
```

A Strict Fold Can Keep Lazy Fields

```haskell
meanPair :: [Double] -> Maybe Double
meanPair [] = Nothing
meanPair xs = Just (total / fromIntegral count)
  where
    (total, count) = foldl' (\(s, n) x -> (s + x, n + 1))
                            (0, 0 :: Int) xs
```

One traversal, but after two steps the pair can contain:

```haskell
((0 + x1) + x2, (0 + 1) + 1)
```

Pitfall

WHNF exposes only the pair constructor.
Both fields can still hold growing chains of delayed additions.

Bang Patterns and `seq`

Enable bang patterns at the top of the source file:

```
{-# LANGUAGE BangPatterns #-}
```

A strict binding demands its value to WHNF before evaluating the body[9]:

```
let !x = e in body
```

[9] GHC contributors. GHC User's Guide. 9.10.3, Section 6.14.1.

Bang Patterns and `seq`

Enable bang patterns at the top of the source file:

```
{-# LANGUAGE BangPatterns #-}
```

A strict binding demands its value to WHNF before evaluating the body[9]:

```
let !x = e in body
```

For a variable binding, this is equivalent to:

```
let x = e in x `seq` body
```

`seq` demands its first argument to WHNF, then returns the second.

[9] GHC contributors. GHC User's Guide. 9.10.3, Section 6.14.1.

Force Both Accumulator Fields

```haskell
stepMean :: (Double, Int) -> Double -> (Double, Int)
stepMean (s, n) x =
  let !s' = s + x
      !n' = n + 1
  in (s', n')
```

Force Both Accumulator Fields



```haskell
stepMean :: (Double, Int) -> Double -> (Double, Int)
stepMean (s, n) x =
  let !s' = s + x
      !n' = n + 1
  in (s', n')
```

```haskell
foldl' stepMean (0, 0) xs
```

Both new numbers are evaluated before the pair is returned.

Force Both Accumulator Fields

```haskell
stepMean :: (Double, Int) -> Double -> (Double, Int)
stepMean (s, n) x =
  let !s' = s + x
      !n' = n + 1
  in (s', n')
```

```haskell
foldl' stepMean (0, 0) xs
```

Both new numbers are evaluated before the pair is returned.

Pitfall

A binding `let !(s, n) = e in body` forces only the pair constructor. The fields still need their own strictness annotations.

A Mean with Explicit Recursion



```haskell
meanRecursive :: [Double] -> Maybe Double
meanRecursive = go 0 0
  where
    go :: Double -> Int -> [Double] -> Maybe Double
    go !s !n xs = case xs of
      [] -> if n == 0 then Nothing
            else Just (s / fromIntegral n)
      x:rest -> go (s + x) (n + 1) rest
```

A Mean with Explicit Recursion

```haskell
meanRecursive :: [Double] -> Maybe Double
meanRecursive = go 0 0
  where
    go :: Double -> Int -> [Double] -> Maybe Double
    go !s !n xs = case xs of
      [] -> if n == 0 then Nothing
            else Just (s / fromIntegral n)
      x:rest -> go (s + x) (n + 1) rest
```

The bangs force both accumulator values on entry to `go`.
Consumed cells can be reclaimed if no other consumer retains the input.

A Mean with Explicit Recursion

```haskell
meanRecursive :: [Double] -> Maybe Double
meanRecursive = go 0 0
  where
    go :: Double -> Int -> [Double] -> Maybe Double
    go !s !n xs = case xs of
      [] -> if n == 0 then Nothing
            else Just (s / fromIntegral n)
      x:rest -> go (s + x) (n + 1) rest
```

The bangs force both accumulator values on entry to `go`.
Consumed cells can be reclaimed if no other consumer retains the input.

Run `lecture-9-mean.hs` to compare the four versions.

Strictness Still Does Not Stop the Traversal



```haskell
foldl' (&&) True [False, True, True]
```

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
```

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
= foldl' (&&) False [True]
```

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
= foldl' (&&) False [True]
= foldl' (&&) False []
```

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
= foldl' (&&) False [True]
= foldl' (&&) False []
= False
```

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
= foldl' (&&) False [True]
= foldl' (&&) False []
= False
```

The accumulator is already `False`, but the fold must find `[]`.

Strictness Still Does Not Stop the Traversal



```
foldl' (&&) True [False, True, True]
= foldl' (&&) False [True, True]
= foldl' (&&) False [True]
= foldl' (&&) False []
= False
```

The accumulator is already `False`, but the fold must find `[]`.

```
foldl' (&&) True (False : undefined)   -- error
```

The elements after `False` need not be evaluated.
The remaining spine still must be traversed.

The Right Fold

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (+) 0 [1, 2, 3]
```

The Right Fold

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (+) 0 [1, 2, 3]
= 1 + foldr (+) 0 [2, 3]
```

The Right Fold

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (+) 0 [1, 2, 3]
= 1 + foldr (+) 0 [2, 3]
= 1 + (2 + foldr (+) 0 [3])
```

The Right Fold

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```
foldr (+) 0 [1, 2, 3]
= 1 + foldr (+) 0 [2, 3]
= 1 + (2 + foldr (+) 0 [3])
= 1 + (2 + (3 + foldr (+) 0 []))
```

The Right Fold

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```
foldr (+) 0 [1, 2, 3]
= 1 + foldr (+) 0 [2, 3]
= 1 + (2 + foldr (+) 0 [3])
= 1 + (2 + (3 + foldr (+) 0 []))
= 1 + (2 + (3 + 0))
```

The Right Fold



```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (+) 0 [1, 2, 3]
= 1 + foldr (+) 0 [2, 3]
= 1 + (2 + foldr (+) 0 [3])
= 1 + (2 + (3 + foldr (+) 0 []))
= 1 + (2 + (3 + 0))
```

“Right” describes the nesting.
Whether the recursive result is needed depends on `f`.

The Right Fold Can Short-Circuit



```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (&&) True (False : undefined)
```

The Right Fold Can Short-Circuit



```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (&&) True (False : undefined)
= False && foldr (&&) True undefined
```

The Right Fold Can Short-Circuit

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (&&) True (False : undefined)
= False && foldr (&&) True undefined
= False
```

The Right Fold Can Short-Circuit

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (&&) True (False : undefined)
= False && foldr (&&) True undefined
= False
```

`(&&)` can ignore its second argument.
The recursive call is therefore not evaluated.

The Right Fold Can Short-Circuit

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
```

```haskell
foldr (&&) True (False : undefined)
= False && foldr (&&) True undefined
= False
```

`(&&)` can ignore its second argument.
The recursive call is therefore not evaluated.

`foldr (&&) True (False : repeat True)` also returns `False`.

The Right Fold Can Produce a Stream

`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
```

The Right Fold Can Produce a Stream



`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
= take 2 (2 : map (*2) [2..])
```

The Right Fold Can Produce a Stream



`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
= take 2 (2 : map (*2) [2..])
= 2 : take 1 (map (*2) [2..])
```

The Right Fold Can Produce a Stream

`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
= take 2 (2 : map (*2) [2..])
= 2 : take 1 (map (*2) [2..])
= 2 : 4 : take 0 (map (*2) [3..])
```

The Right Fold Can Produce a Stream

`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
= take 2 (2 : map (*2) [2..])
= 2 : take 1 (map (*2) [2..])
= 2 : 4 : take 0 (map (*2) [3..])
= [2, 4]
```

The Right Fold Can Produce a Stream



`(:)` can return a constructor without evaluating its tail.

```haskell
map :: (a -> b) -> [a] -> [b]
map f = foldr (\x rest -> f x : rest) []
```

```haskell
take 2 (map (*2) ([1..] :: [Integer]))
= take 2 (2 : map (*2) [2..])
= 2 : take 1 (map (*2) [2..])
= 2 : 4 : take 0 (map (*2) [3..])
= [2, 4]
```

We can demand output before reaching the end of the input.

Choosing a Fold

For lists, separate association from strictness[10].

	Left-associated	Right-associated
Lazy	`foldl` Usually avoid on lists. May build a thunk chain.	Recommended: `foldr` Can short-circuit. Can produce a stream.
Strict	Recommended: `foldl'` Use for a finite reduction. Accumulator forced to WHNF.	No streaming benefit. Must reach the end first. Needs linear auxiliary space.

[10] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: choosing folds.

Choosing a Fold

For lists, separate association from strictness[10].

	Left-associated	Right-associated
Lazy	`foldl` Usually avoid on lists. May build a thunk chain.	Recommended: `foldr` Can short-circuit. Can produce a stream.
Strict	Recommended: `foldl'` Use for a finite reduction. Accumulator forced to WHNF.	No streaming benefit. Must reach the end first. Needs linear auxiliary space.

A lazy fold's step still decides what is demanded.

[10] GHC contributors. Data.Foldable. base 4.20.0.0, 2024: choosing folds.

Question from Quiz 8: What Does the Loop Retain?



```haskell
countLines :: Int -> IO ()
countLines n = do
  s <- getLine
  if s == "quit" then print n else do
    let next = n + 1
    next `seq` countLines next
```

Question from Quiz 8: What Does the Loop Retain?

```haskell
countLines :: Int -> IO ()
countLines n = do
  s <- getLine
  if s == "quit" then print n else do
    let next = n + 1
    next `seq` countLines next
```

The call receives an evaluated counter, not another delayed addition. After testing `s`, the loop does not keep that input line.

Question from Quiz 8: What Does the Loop Retain?

```haskell
countLines :: Int -> IO ()
countLines n = do
  s <- getLine
  if s == "quit" then print n else do
    let next = n + 1
    next `seq` countLines next
```

The call receives an evaluated counter, not another delayed addition. After testing `s`, the loop does not keep that input line.

The parcel clerk in topic 8 keeps the stored parcels for later commands. That list can grow even when the counter and call stack stay bounded.

Quick Recap

1. What does matching `Mul a b` demand?
2. Why does `take 3 (from 1)` finish?
3. Does `foldr` always process a whole list from the right?
4. Does `foldl'` stop when its result is already decided?
5. Does `foldl'` force the fields of a pair?

Quick Recap

1. What does matching `Mul a b` demand?
   The outer constructor, but not the fields.
2. Why does `take 3 (from 1)` finish?
3. Does `foldr` always process a whole list from the right?
4. Does `foldl'` stop when its result is already decided?
5. Does `foldl'` force the fields of a pair?

Quick Recap

1. What does matching `Mul a b` demand?
   The outer constructor, but not the fields.
2. Why does `take 3 (from 1)` finish?
   It asks for only three cells, each produced in finite time.
3. Does `foldr` always process a whole list from the right?
4. Does `foldl'` stop when its result is already decided?
5. Does `foldl'` force the fields of a pair?

Quick Recap

1. What does matching `Mul a b` demand?
   The outer constructor, but not the fields.
2. Why does `take 3 (from 1)` finish?
   It asks for only three cells, each produced in finite time.
3. Does `foldr` always process a whole list from the right?
   The step decides whether to demand the recursive result.
4. Does `foldl'` stop when its result is already decided?

5. Does `foldl'` force the fields of a pair?

Quick Recap

1. What does matching `Mul a b` demand?
   The outer constructor, but not the fields.
2. Why does `take 3 (from 1)` finish?
   It asks for only three cells, each produced in finite time.
3. Does `foldr` always process a whole list from the right?
   The step decides whether to demand the recursive result.
4. Does `foldl'` stop when its result is already decided?
   No. It still traverses the whole spine.
5. Does `foldl'` force the fields of a pair?

Quick Recap

1. What does matching `Mul a b` demand?
   The outer constructor, but not the fields.
2. Why does `take 3 (from 1)` finish?
   It asks for only three cells, each produced in finite time.
3. Does `foldr` always process a whole list from the right?
   The step decides whether to demand the recursive result.
4. Does `foldl'` stop when its result is already decided?
   No. It still traverses the whole spine.
5. Does `foldl'` force the fields of a pair?
   No. Force the fields explicitly if they accumulate delayed work.

Quiz for Lecture 9

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 9

Ten minutes, on Moodle, now.
Seven questions on this lecture, then three about the lecture itself.

No AI. The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

Say what you learned and what stayed unclear, **specifically**.
I read them before writing the next lecture.

What's Next

In the topic 9 lab you will

- write the worksheet on lazy evaluation;
- take part in the scheduled homework defenses;
- practise programming with lazy lists and trees.

→ **Before Topic 10**

Load `lecture-9.hs` and run `demoPipeline` in GHCi.
Topic 10 develops a board-game AI with lazy game trees.

Thank you!



References

- Hughes, J. (1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2, pp. 98–107. DOI: 10.1093/comjnl/32.2.98. URL: https://doi.org/10.1093/comjnl/32.2.98.
- Kiselyov, Oleg (2023). ***Simple Generators in OCaml.*** Section of Simple Generators: Alternative to Lazy Evaluation. URL: https://okmij.org/ftp/continuations/PPYield/index.html#gen-OCaml (visited on 10/20/2026).
- Marlow, Simon (2010). ***Haskell 2010 Language Report.*** URL: https://www.haskell.org/onlinereport/haskell2010/ (visited on 09/27/2026).

References ii

- The GHC Team (2024a). ***GHC User's Guide, version 9.10.*** URL: https://downloads.haskell.org/ghc/9.10.3/docs/users_guide/ (visited on 09/27/2026).
- — (2024b). ***The base Library, Version 4.20.0.0.*** URL: https://hackage.haskell.org/package/base-4.20.0.0 (visited on 10/20/2026).
- Xia, Li-yao et al. (2024). **"Story of Your Lazy Function's Life: A Bidirectional Demand Semantics for Mechanized Cost Analysis of Lazy Programs".** In: *Proceedings of the ACM on Programming Languages* 8.ICFP, 237. DOI: 10.1145/3674626. URL: https://doi.org/10.1145/3674626.